Вариант №75

Параметры варианта

Ось	Значение	Используется
Приложение	notes-py (Python)	ЛР2–ЛР8
Git-платформа	GitLab CE	ЛР2–ЛР8
Сканер образа	Trivy	ЛР3–ЛР4
Движок конвейера	GitLab CI	ЛР4–ЛР8
Дистрибутив Kubernetes	k3s	ЛР6–ЛР8
Инструмент доставки	Argo CD	ЛР7–ЛР8
Хранилище метрик и логов	VictoriaMetrics и VictoriaLogs	ЛР8
Сбор сигналов	штатные агенты стека	ЛР8

Таблица 1.1: Параметры варианта.

Вариант выдаётся на семестр и не меняется; замена — по согласованию с преподавателем. Работа, выполненная на других инструментах, не принимается. Справочные разделы — в методических указаниях.

Вместо учебного приложения по согласованию с преподавателем допускается собственный проект при выполнении тех же требований; соответствие проверяется при сдаче ЛР2.

Лабораторная работа 1. Стенд курса и модель угроз

Цель

Развернуть стенд курса, зафиксировать исходную модель угроз и инструкцию для ИИ-агента.

Исходные данные

Разделы методических указаний: «Стенд курса» — требования, порядок развёртывания, файл первичной настройки и проверка; «Модель угроз» — формат и исходная таблица `THREATS.md`; «Инструкция агенту» — образец `AGENTS.md`.

Предусловия

- на рабочей машине установлены `ssh` и `git`;
- установлен гипервизор с поддержкой cloud-init;
- создана пара ключей для входа на узлы:

```
ssh-keygen -t ed25519 -C "student@laptop" -f ~/.ssh/devops
```

Задание

Стенд

1. Создайте две виртуальные машины по требованиям из раздела «Стенд курса» методических указаний. Обе настраиваются файлом первичной настройки оттуда же: подставляются имя узла, открытый ключ и хеш пароля. На `node-2` порты 2222, 8443, 5050 и 6443 не открываются.
2. Открытый ключ и хеш пароля подставляются при сборке ISO. В репозиторий попадает файл с плейсхолдерами: хеш пароля — секрет и в репозитории не хранится. Открытый ключ секретом не является, но и он остаётся плейсхолдером, чтобы файл настройки был одинаков у всех.
3. Пропишите имена узлов и имена окружений на рабочей машине и на обоих узлах — строками из раздела «Стенд курса».
4. Выполните проверку стенда из раздела «Стенд курса» методических указаний. Каждая команда должна дать описанный там результат; после первого входа ручных шагов настройки быть не должно.

Репозиторий инфраструктуры

5. Заведите локальный репозиторий `infra` и добавьте в него файлы первичной настройки обоих узлов. Отправка репозитория на платформу — в ЛР2.
6. Заполните `THREATS.md` по разделу «Модель угроз» методических указаний: перенесите исходную таблицу одним блоком, добавьте не менее двух собственных угроз своего стенда и переведите строку T1 в статус `закрыто`, указав в колонке «Где реализовано» файл или настройку.
7. Заведите `AGENTS.md` по разделу «Инструкция агенту» методических указаний: инварианты стенда, команды проверки, запреты.

Точка возврата

8. Сделайте снимок состояния обеих машин и укажите дату. Снимок обновляется после каждой сданной работы.

Приёмка

- [ ] **показывается на защите** — стенд проходит проверку из раздела «Стенд курса» методических указаний: вход на оба узла по ключу, парольный вход отклоняется, узлы видят друг друга по именам, а порт, которого нет в таблице для этого узла, с рабочей машины закрыт;

- [ ] **показывается на защите** — на обоих узлах заведены пользователь с `sudo` и пользователь приложения без `sudo`; `docker run --rm hello-world` работает без `sudo`;
- [ ] **проверяется по файлам** — оба узла созданы из файла первичной настройки по образцу методических указаний, после первой загрузки ручных шагов настройки не было;
- [ ] **проверяется по файлам** — в репозитории `infra` лежат файлы первичной настройки обоих узлов, `THREATS.md` (исходная таблица и не менее двух собственных угроз) и `AGENTS.md` с разделами «инварианты стенда», «команды проверки», «запреты»;
- [ ] **проверяется по файлам** — снимок состояния обеих машин существует и датирован днём сдачи;
- [ ] **проверяется по файлам** — строка T1 таблицы угроз закрыта;
- [ ] объяснить принятые решения и повторить любой шаг по требованию.

Лабораторная работа 2. Git-платформа и процесс

Цель

Развернуть Git-платформу варианта на своём узле с registry по TLS от собственной CA, организовать репозитории с защищённой веткой, ревью и релизным тегом, поставить проверку секретов до коммита.

Исходные данные

- платформа варианта — **GitLab CE**;
- порядок развёртывания, создание собственной CA, пример на Compose и первый вход — раздел «Git-платформа» методических указаний;
- репозиторий учебного приложения:

`https://github.com/nitwof/devops-notes-py` — «Заметки», Python.

Предусловия

- стенд ЛР1 работает, снимок состояния актуален;
- репозиторий `infra` заведён локально.

Задание

Собственная CA и платформа

1. Создайте собственную CA и сертификат `node-1` по разделу «Git-платформа» методических указаний. Ключ CA остаётся на `node-1` и в репозиторий не попадает. Добавьте CA в доверенные на обоих узлах и на рабочей машине.
2. Разверните платформу варианта. Способ произволен; проверяется результат: интерфейс открывается по HTTPS на `node-1:8443` без предупреждений браузера, SSH Git отвечает на 2222 (платформа в контейнере) или на 22 (платформа установлена пакетом на хост).
3. Включите registry платформы и убедитесь через интерфейс, что он требует аутентификацию. Адрес registry варианта — **node-1:5050**. Клиент `docker` и публикация образов — в ЛР3.

Репозитории и процесс

4. Создайте приватную организацию (в GitLab — группу): образы публичной организации registry отдаёт без входа. Импортируйте в неё репозиторий учебного приложения с историей: зеркальный клон и push в созданный на платформе репозиторий либо штатный импорт по URL. История и все коммиты должны сохраниться.
5. Отправьте на платформу репозиторий `infra`.
6. Защитите основную ветку обоих репозиториев: прямой push запрещён, изменение идёт через запрос на слияние.
7. Заведите вторую учётную запись и назначьте её ревьюером. Стенд индивидуальный, поэтому ревьюер — вторая учётная запись самого студента; свободная регистрация на платформе выключена, учётная запись заводится администратором. Проведите через запрос на слияние изменение в репозитории приложения (например, правку README): ветка, описание, комментарий ревьюера, одобрение или разрешённое обсуждение — способ зависит от платформы. Слияние выполняется после ревью.
8. Поставьте на основную ветку репозитория приложения аннотированный тег `v0.1.0`.

Секреты

9. Установите в обоих репозиториях хук до коммита, который ищет секреты (`gitleaks` или аналог с конфигурацией в репозитории). Проверьте его на тестовом секрете: коммит должен быть отклонён. Тестовый секрет в историю не попадает.

10. Опишите порядок ротации утёкшего секрета разделом «Ротация секрета» под таблицей `THREATS.md` и закройте строки T2 и T3.

Дополнительное задание

Настройте подпись коммитов ключом SSH или GPG, зарегистрируйте открытый ключ подписи в профиле платформы и добейтесь, чтобы платформа показывала коммиты проверенными.

Приёмка

Основное задание:

- [ ] **показывается на защите** — платформа доступна по HTTPS на `node-1:8443` с сертификатом собственной CA (без отключения проверки на клиенте) и отвечает по SSH;
- [ ] **показывается на защите** — хук до коммита блокирует тестовый секрет; студент называет порядок ротации утёкшего секрета;
- [ ] **проверяется по файлам** — registry платформы включён и требует аутентификацию, проверка выполнена через интерфейс платформы;
- [ ] **проверяется по файлам** — репозиторий приложения импортирован с историей, репозиторий `infra` отправлен на платформу;
- [ ] **проверяется по файлам** — основная ветка защищена, изменение прошло через запрос на слияние с ревью, на основной ветке стоит аннотированный тег `v0.1.0`;
- [ ] **проверяется по файлам** — строки T2 и T3 таблицы угроз закрыты;
- [ ] объяснить принятые решения и повторить любой шаг по требованию.

Дополнительное задание:

- [ ] **проверяется по файлам** — коммиты подписаны, платформа показывает их проверенными.

Лабораторная работа 3. Контейнеризация приложения

Цель

Собрать образ приложения по лучшим практикам, поднять стек одной командой Compose, опубликовать образ в свой registry и проверить его сканером и SBOM.

Исходные данные

- сканер варианта — **Trivy**;
- registry платформы — **node-1:5050**, включён в ЛР2;
- требования к приложению (переменные окружения, служебные эндпоинты, команда миграции) — README импортированного репозитория.

Предусловия

- платформа и registry ЛР2 работают;
- CA доверена на обоих узлах и на рабочей машине;
- движок Docker на рабочей машине не нужен: сборка идёт на `node-1`.

Задание

Образ и стек

1. Заведите контекст Docker на `node-1` и сделайте его текущим:

```
docker context create node1 --docker "host=ssh://student@node-1"
docker context use node1
```

2. Заведите адрес registry и полное имя образа в переменные окружения: имя понадобится сборке, Compose и публикации.

```
export REG=node-1:5050
export IMG=$REG/<владелец>/<приложение>
```

3. Напишите `Dockerfile` и `.dockerignore` в репозитории приложения. Требования: многостадийная сборка; базовый образ закреплён по digest; процесс в итоговом образе работает не от root, `USER` задан числом (UID) — иначе кластер ЛР6 не проверит, что пользователь не root; `HEALTHCHECK` обращается к `/healthz`; версия и короткий SHA коммита передаются аргументами сборки и видны в `/version`.

   Базовый образ — `python:3.13-slim`. Зависимости ставятся из `uv.lock` командой `uv sync --frozen --no-dev` на отдельной стадии; в итоговую стадию копируются только окружение и код, `uv` в неё не попадает. Размер итогового образа — не более 300 МБ.

4. Соберите образ, передав версию и короткий SHA коммита аргументами сборки, и измерьте его размер как сумму слоёв. Колонка `DISK USAGE` вывода `docker images` прибавляет к слоям их сжатое содержимое в хранилище, и порог задания по ней не проверяется:

```
docker build -t $IMG:v0.1.0 \
  --build-arg APP_VERSION=v0.1.0 \
  --build-arg APP_COMMIT=$(git rev-parse --short HEAD) .
docker history -H=false --format '{{.Size}}' $IMG:v0.1.0 \
  | awk '{s+=$1} END {print s/1e6 " MB"}'
```

5. Напишите `compose.yaml`: приложение, Postgres 17 и Valkey 8. Приложение запускается из собранного образа `$IMG:v0.1.0`. У Postgres и Valkey — healthcheck, у приложения — зависимость с условием готовности. Приложение публикуется на `node-1:80`, данные Postgres лежат в именованном томе, значения окружения — в `.env` вне репозитория (в репозиторий идёт `.env.example`).
6. Примените миграцию отдельным запуском контейнера (`docker compose run --rm`), а не при старте приложения. Проверьте идемпотентность: второй прогон завершается успехом и ничего не меняет.
7. Поднимите стек одной командой и проверьте ответ приложения. Эндпоинт `/version` обязан вернуть версию и SHA, переданные при сборке:

```
docker compose up -d
curl -fsS http://node-1/ ; curl -fsS http://node-1/readyz
curl -fsS http://node-1/version
```

Публикация и проверка образа

8. Войдите в registry и опубликуйте образ. Отключение проверки сертификата (`insecure-registries`) не допускается: registry работает по TLS собственной CA. Скачивание обратно проверяет, что образ действительно опубликован; стек перед удалением образа останавливается, потому что образ работающего контейнера движок не удаляет:

```
docker login $REG
docker push $IMG:v0.1.0
docker compose down
docker rmi $IMG:v0.1.0
docker pull $IMG:v0.1.0
```

9. Проверьте опубликованный образ сканером варианта и сформируйте SBOM. Сканер работает с рабочей машины и обращается к registry напрямую — движок Docker ему не нужен, достаточно доверенной CA из ЛР2.

```
trivy image --exit-code 1 --severity HIGH,CRITICAL $IMG:v0.1.0
trivy image --format cyclonedx --output sbom.json $IMG:v0.1.0
```

10. Уровни HIGH и CRITICAL устраняются обновлением базового образа и зависимостей. Оставшиеся находки перечисляются в файле исключений с причиной и сроком пересмотра; отчёт сканера и SBOM хранятся в репозитории приложения.
11. Закройте строки T4 и T5a таблицы угроз.

Дополнительное задание

Соберите образ для `amd64` и `arm64` одним манифестом. Сборка идёт на `node-1`, поэтому эмуляция включается на самом узле. Отдельный сборщик (`docker buildx create`) не заводится: он работает в своём контейнере и не видит CA стенда, публикация завершается ошибкой `x509`. Штатный сборщик движка собирает обе архитектуры сам:

```
docker run --privileged --rm tonistiigi/binfmt --install arm64
docker buildx build --platform linux/amd64,linux/arm64 -t $IMG:v0.1.0 --push .
```

Приёмка

Основное задание:

- [ ] **показывается на защите** — `docker compose up -d` поднимает приложение, Postgres и Valkey с healthcheck и условиями готовности, приложение отвечает на `node-1:80`;
- [ ] **показывается на защите** — вход в registry выполняется, образ публикуется по TLS собственной CA и скачивается обратно;
- [ ] **проверяется по файлам** — образ собирается многостадийно, процесс в итоговом образе работает не от root, имеет `HEALTHCHECK`, базовый образ закреплён по digest, размер (сумма слоёв) не превышает порог задания;
- [ ] **проверяется по файлам** — миграция выполняется отдельной командой и идемпотентна;
- [ ] **проверяется по файлам** — отчёт сканера варианта не содержит уровней HIGH и CRITICAL либо каждое исключение обосновано; SBOM сформирован и лежит в репозитории;
- [ ] **проверяется по файлам** — строки T4 и T5a таблицы угроз закрыты;
- [ ] объяснить принятые решения и повторить любой шаг по требованию.

Дополнительное задание:

- [ ] **проверяется по файлам** — манифест образа в registry содержит `amd64` и `arm64`.

Лабораторная работа 4. CI и DevSecOps

Цель

Построить конвейер на своей платформе с собственным runner'ом и проверками lint, test, build, scan, publish и sign, публикующий подписанные образы без секретов в репозитории.

Исходные данные

- движок конвейера варианта — **GitLab CI**;
- сканер — **Trivy**, registry — **node-1:5050**;
- команды линтера, тестов, сборки и миграции — цели `Makefile` репозитория приложения.

Предусловия

- образ ЛР3 собирается и публикуется вручную;
- стек Compose работает;
- все шаги конвейера выполняются на `node-1`; пара ключей подписи создаётся один раз на рабочей машине (шаг 9).

Задание

Runner

1. Установите на `node-1` runner варианта и зарегистрируйте его на платформе.
   Runner — `gitlab-runner`, регистрация по authentication token, созданному в настройках проекта или группы (устаревшая регистрация по registration token не используется); executor — Docker. Конвейер описывается файлом `.gitlab-ci.yml`.

Конвейер

2. Опишите конвейер с шестью проверками: `lint`, `test`, `build`, `scan`, `publish`, `sign`. Каждый шаг вызывает ту же команду, что и локальная цель `Makefile`.

   Образам шагов с объявленной точкой входа (сканер, поиск секретов) задаётся `entrypoint: [""]`: иначе скрипт job'а уходит аргументами этой команды.
3. Настройте триггеры: `lint` и `test` выполняются на каждый push и на каждый запрос на слияние; `build`, `scan`, `publish` и `sign` — только на основной ветке и на релизном теге.
4. Интеграционный тест запускается с настоящей базой данных: Postgres поднимается сервисом job'а, адрес передаётся переменной `TEST_DATABASE_URL`.
5. Шаг `build` собирает образ внутри job'а, а job сам работает в контейнере — значит доступ к сборке нужно дать явно. Executor у всех runner'ов курса — Docker; выбирается способ доступа к сборке. Выберите один и обоснуйте выбор на защите:
   - **проброс сокета движка узла** (`/var/run/docker.sock` в job) — самый простой и самый опасный: job получает права root на `node-1`, поэтому такой выбор добавляется в `THREATS.md` собственной строкой;
   - **отдельный движок в контейнере** (docker-in-docker сервисом job'а) — изоляция есть, но контейнер привилегированный и кеш слоёв теряется между запусками;
   - **сборщик без движка** (rootless BuildKit или `buildah`) — привилегий не нужно, зато синтаксис сборки и кеш отличаются от `docker build`.
6. Публикуйте образ с двумя тегами: коротким SHA коммита и, на релизном теге, `vX.Y.Z`. Внутри job'а registry работает по TLS собственной CA — сертификат монтируется в job и добавляется в доверенные.

Безопасность конвейера

7. Вынесите учётные данные registry и подписи в переменные CI с включённым маскированием. Секретов в репозитории быть не должно; добавьте в конвейер шаг поиска секретов на каждый push, до сборки, и добейтесь чистого прогона.

   Учётные данные registry платформы GitLab выдаёт job'у сам на время прогона — переменные `CI_REGISTRY_USER` и `CI_REGISTRY_PASSWORD`; заводить для них переменные CI не нужно.

8. Закрепите внешние зависимости конвейера: сторонние шаги — по SHA коммита, образы шагов — по digest. Плавающие теги (`latest`, `v1`) в конвейере не используются.

9. Подпишите опубликованный образ и проверьте подпись в том же конвейере — на основной ветке и на релизном теге. Пару ключей создайте на рабочей машине: приватный ключ хранится переменной CI платформы и на диск `node-1` не попадает; job получает его в окружении на время прогона.

```
cosign generate-key-pair                    # на рабочей машине, один раз
cosign sign --key env://COSIGN_PRIVATE_KEY $IMG@<digest>
cosign verify --key cosign.pub $IMG@<digest>
```

   Приватный ключ и пароль к нему хранятся переменными CI, открытый ключ — в репозитории. Вывод проверки остаётся в логе конвейера.

10. Закройте строки T6, T7 и T8a таблицы угроз.

Дополнительное задание

Добавьте в конвейер публикацию provenance-аттестации сборки (`cosign attest` или штатный механизм движка) и её проверку отдельным шагом: с аттестацией конвейер достигает уровня SLSA Build L1.

Приёмка

Основное задание:

- [ ] **показывается на защите** — push в репозиторий запускает конвейер на runner'е варианта;
- [ ] **показывается на защите** — отчёт тестов и результат проверки подписи открываются со страницы прогона: отчёт — вкладкой тестов (GitLab CI), артефактом прогона (Actions) или журналом шага (Woodpecker), проверка подписи — журналом шага; студент объясняет выбранный способ доступа job'а к сборке образа и чем этот способ платит;
- [ ] **проверяется по файлам** — `lint` и `test` выполняются на каждый push и запрос на слияние, а `build`, `scan`, `publish` и `sign` — только на основной ветке и релизном теге;
- [ ] **проверяется по файлам** — образы помечены коротким SHA коммита и тегом `vX.Y.Z`;
- [ ] **проверяется по файлам** — секреты хранятся в переменных CI и маскируются, поиск секретов в конвейере чист, в репозитории секретов нет;
- [ ] **проверяется по файлам** — внешние шаги конвейера закреплены по SHA, образы шагов — по digest;
- [ ] **проверяется по файлам** — подпись опубликованного образа проверяется шагом конвейера, вывод проверки есть в логе;
- [ ] **проверяется по файлам** — строки T6, T7 и T8a таблицы угроз закрыты;
- [ ] объяснить принятые решения и повторить любой шаг по требованию.

Дополнительное задание:

- [ ] **проверяется по файлам** — конвейер публикует provenance-аттестацию сборки, и она проверяется отдельным шагом.

Лабораторная работа 5. Инфраструктура как код

Цель

Описать конфигурацию обоих узлов и runner'а ролями Ansible с доказанной идемпотентностью и управлять объектами Git-платформы через OpenTofu с обнаружением дрейфа.

Исходные данные

- платформа варианта — **GitLab CE**, движок конвейера — **GitLab CI**;
- в код репозитория `infra` переносятся настройка узлов из ЛР1 (пользователи, SSH, firewall, Docker), доверие к CA и runner из ЛР2 и ЛР4, объекты Git-платформы из ЛР2; развёртывание платформы и записи `hosts` остаются по разделам «Стенд курса» и «Git-платформа».

Предусловия

- стенд, платформа, registry и конвейер работают;
- снимок состояния узлов актуален: код применяется к работающему стенду, и откат должен быть возможен.

Задание

Конфигурация узлов ролями Ansible

1. Соберите проект Ansible в репозитории `infra`: инвентарь с двумя узлами, `ansible.cfg`, `site.yml` и роли `users`, `ssh`, `firewall`, `docker`, `ca`, `runner`. Модули вызываются полными именами (FQCN), перезапуск служб — через handlers, команды `shell` и `command` — только там, где штатного модуля нет, и с условием повторного запуска.
2. Роль `ca` раскладывает корневой сертификат курса на оба узла, обновляет хранилище доверенных сертификатов и перезапускает Docker при изменении файла.
3. Роль `runner` описывает установку и регистрацию runner'а варианта на `node-1`. Токен регистрации приходит переменной, в код не пишется.
4. Докажите идемпотентность: второй прогон `ansible-playbook site.yml` даёт ноль изменённых задач. Прогон `--check` на неизменённом стенде тоже показывает ноль изменений.
5. Добейтесь чистого прогона `ansible-lint`.

Объекты платформы в OpenTofu

6. Опишите объекты Git-платформы кодом OpenTofu в каталоге `tofu/`: организация, оба репозитория, защита основной ветки, переменные CI.

   Провайдер — `gitlabhq/gitlab`. Существующие объекты вносятся в состояние командой `tofu import`, а не пересоздаются.

7. Входные значения — токен провайдера и значения переменных CI — приходят переменными окружения `TF_VAR_*`, а не файлом в репозитории. Значения хранятся в зашифрованном файле Ansible Vault и передаются так:

```
export TF_VAR_platform_token="$(ansible-vault view secrets.yml | yq '.platform_token')"
tofu plan
```

Состояние и дрейф

8. Храните state вне репозитория.

   Допустимы оба способа: HTTP-backend платформы или локальный файл вне репозитория с правами `600`.

   State содержит значения переменных CI в открытом виде, поэтому в Git он не попадает ни при каких условиях.

9. Проверьте обнаружение дрейфа: измените защиту ветки или описание репозитория руками в интерфейсе платформы и убедитесь, что `tofu plan` показывает расхождение. Верните состояние командой `tofu apply`.

10. Закройте строки T9 и T10a таблицы угроз.

Дополнительное задание

Опишите кодом OpenTofu создание виртуальной машины `node-2` через провайдер своей площадки (Proxmox, libvirt или VirtualBox) и пересоздайте узел: машина поднимается из кода, первичная настройка отрабатывает файл, записи `hosts` вносятся по разделу «Стенд курса», playbook приводит узел к рабочему состоянию.

Приёмка

Основное задание:

- [ ] **показывается на защите** — `ansible-playbook site.yml` применяется к обоим узлам, второй прогон даёт ноль изменений;
- [ ] **показывается на защите** — после ручной правки в интерфейсе платформы `tofu plan` показывает дрейф;
- [ ] **проверяется по файлам** — `ansible-lint` чист, роли разделены по назначению, перезапуски идут через handlers;
- [ ] **проверяется по файлам** — корневой сертификат курса доверен на обоих узлах после применения роли;
- [ ] **проверяется по файлам** — OpenTofu управляет организацией, репозиториями, защитой ветки и переменными CI;
- [ ] **проверяется по файлам** — state лежит вне Git, секреты в репозитории зашифрованы, входные значения приходят переменными окружения;
- [ ] **проверяется по файлам** — строки T9 и T10a таблицы угроз закрыты;
- [ ] объяснить принятые решения и повторить любой шаг по требованию.

Дополнительное задание:

- [ ] **проверяется по файлам** — `node-2` пересоздан кодом OpenTofu через провайдер площадки и приведён в рабочее состояние playbook'ом.

Лабораторная работа 6. Оркестрация

Цель

Развернуть двухузловой кластер Kubernetes кодом и перенести в него приложение манифестами с пробами, ресурсами, конфигурацией, базой данных в StatefulSet и миграцией в Job, показав обновление без простоя.

Исходные данные

- дистрибутив варианта — **k3s**;
- входной шлюз, класс хранилища и способ доверия к собственной CA одинаковы для всех дистрибутивов: Traefik как контроллер Gateway API, `local-path-provisioner` как класс хранилища по умолчанию;
- registry — **node-1:5050**.

Предусловия

- стенд описан кодом (ЛР5), конвейер публикует подписанные образы (ЛР4);
- кластер занимает порты 80 и 443 на обоих узлах, поэтому перед установкой остановите стек Compose и убедитесь, что порты свободны:

```
docker compose down
ss -tlnp | grep -E ':(80|443)\b'
```

Задание

Кластер

1. Опишите установку кластера ролью Ansible в репозитории `infra`. Роль ставит дистрибутив варианта на оба узла и доводит кластер до состояния, в котором оба узла `Ready`, а control plane принимает рабочую нагрузку. Файл kubeconfig роль копирует с `node-1` на рабочую машину (`~/.kube/config`, адрес API — `node-1`); все команды `kubectl` в работе выполняются с рабочей машины.

   Сервер ставится на `node-1`, агент — на `node-2`. Встроенные Traefik и `local-storage` отключаются при установке (`--disable=traefik`, `--disable=local-storage`): шлюз и класс хранилища ставятся отдельно и одинаково для всех дистрибутивов. Рабочая нагрузка на control plane разрешена по умолчанию.

2. Настройте среду выполнения контейнеров обоих узлов на доверие к собственной CA, иначе образы из своего registry не скачаются. Настройка описывается той же ролью.

3. Запускайте роль кластера заданием конвейера репозитория `infra`: `ansible-lint` — на каждый запрос на слияние, `ansible-playbook site.yml` — на основной ветке. Для прогона из конвейера заведите отдельную пару ключей SSH: приватный ключ и пароль Ansible Vault хранятся переменными CI с маскированием, открытый ключ выдаётся узлам ролью `users` из ЛР5. Ключ студента в конвейер не попадает и отзывается отдельно.

4. Установите входной шлюз и класс хранилища манифестами из документации компонентов: CRD Gateway API, Traefik как контроллер шлюза (DaemonSet с портами 80 и 443 на обоих узлах), `local-path-provisioner` как класс по умолчанию. Канал CRD (standard или experimental) задаёт контроллер: Traefik работает с каналом standard. EntryPoint контроллера слушает тот же порт, что указан в listener'е `Gateway`: по умолчанию entryPoint `web` у Traefik слушает 8000, и `Gateway` с listener'ом на 80 остаётся в состоянии `ListenersNotValid`, — порты задаются аргументами `--entrypoints.*.address` в манифесте DaemonSet.

Приложение в кластере

5. Опишите манифесты приложения в каталоге `k8s/` репозитория `infra`: namespace `dev`, `Deployment` с двумя репликами, `Service`, `Gateway` и `HTTPRoute`. Маршрут отвечает на имена `dev.node-1` и `dev.node-2` из раздела «Стенд курса»: по имени шлюз отличает окружения, а два имени дают доступ через любой узел. У контейнера обязательны:
   - проба готовности на `/readyz` и проба живости на `/healthz`;
   - `requests` и `limits` по памяти и процессору;
   - `securityContext`:
     - `runAsNonRoot` и `readOnlyRootFilesystem` включены;
     - `allowPrivilegeEscalation` выключен;
     - capabilities сброшены (`drop: [ALL]`), профиль seccomp — `RuntimeDefault`.
6. Конфигурация приложения передаётся `ConfigMap`, пароли и строки подключения — `Secret`. В том же `Secret` лежит `ADMIN_TOKEN` — им закрыт служебный эндпоинт приложения, который понадобится в ЛР8. `Secret` и учётные данные registry создаются командой `kubectl create secret` с рабочей машины и в `k8s/` не хранятся: в Git они попадут только запечатанными, в ЛР7. Образ скачивается из своего registry по `imagePullSecrets`. Приложение работает под собственным `ServiceAccount` с минимальными правами; `automountServiceAccountToken` выключен, если токен не нужен.
7. Опишите базу данных и кеш: Postgres — `StatefulSet` с `PersistentVolumeClaim`, Valkey — `Deployment` без тома. Миграция схемы выполняется отдельным `Job` до выката новой версии приложения. Проба готовности проверяет доступность базы, а не применённость схемы: без `Job` приложение готово, а предметные эндпоинты отвечают `500`.
8. Проверьте доступность приложения с рабочей машины через оба узла: маршрут шлюза отвечает и на `http://dev.node-1/`, и на `http://dev.node-2/`.
9. Поставьте на основную ветку репозитория приложения тег `v0.2.0` и дождитесь, пока конвейер ЛР4 опубликует образ с этим тегом.
10. Покажите обновление без простоя: запустите генератор нагрузки `scripts/load.sh` из репозитория приложения, переведите развёртывание на новый тег и убедитесь, что ошибок нет. `IMG` — полное имя образа из ЛР3:

```
./scripts/load.sh http://dev.node-1 120 &
kubectl -n dev set image deploy/app app=$IMG:v0.2.0
kubectl -n dev rollout status deploy/app
kubectl -n dev rollout undo deploy/app
```

11. Закройте строки T5b и T11a таблицы угроз.

Дополнительное задание

Опишите `NetworkPolicy`, запрещающую весь трафик в namespace по умолчанию и разрешающую только обращения приложения и `Job` миграций к базе данных и кешу, разрешение имён через DNS кластера и входящий трафик от шлюза к приложению. Запрет по умолчанию действует на все поды namespace, поэтому правила нужны и хранилищам (вход), и `Job` (выход). Проверьте запрет обращением из постороннего пода. Политику исполняет сетевой плагин: у k3s — встроенный контроллер политик, у k0s — kube-router, у kubeadm — Cilium.

> **Предупреждение.** Политики действуют и на ЛР8: сбор метрик и логов идёт из другого namespace. Если дополнительное задание сдано, в ЛР8 потребуются правила, пропускающие агентов наблюдаемости и опрос метрик, иначе панели останутся пустыми.

Приёмка

Основное задание:

- [ ] **показывается на защите** — обновление на новый тег под непрерывными запросами проходит без ошибок, откат выполняется штатной командой;
- [ ] **показывается на защите** — причина состояния пода, сломанного преподавателем, названа и устранена;
- [ ] **проверяется по файлам** — кластер дистрибутива варианта из двух узлов установлен ролью Ansible, оба узла `Ready`, control plane принимает нагрузку, шлюз отвечает на нужных портах на обоих узлах, класс хранилища по умолчанию назначен;
- [ ] **проверяется по файлам** — роль кластера применяется заданием конвейера репозитория `infra` с отдельным ключом прогона, `ansible-lint` стоит гейтом на запрос на слияние;
- [ ] **проверяется по файлам** — `Deployment` с пробами, ресурсами и `securityContext` без root и со сброшенными capabilities, `Service` и маршрут шлюза доступны с рабочей машины через любой узел;
- [ ] **проверяется по файлам** — конфигурация в `ConfigMap`, секреты в `Secret`, образ скачивается из своего registry по TLS собственной CA, `ServiceAccount` приложения имеет минимальные права;
- [ ] **проверяется по файлам** — Postgres работает как `StatefulSet` с `PersistentVolumeClaim`, Valkey — как `Deployment`, миграция выполнена `Job`;
- [ ] **проверяется по файлам** — строки T5b и T11a таблицы угроз закрыты;
- [ ] объяснить принятые решения и повторить любой шаг по требованию.

Дополнительное задание:

- [ ] **проверяется по файлам** — `NetworkPolicy` по умолчанию запрещает трафик и разрешает только приложение и `Job` миграций к базе данных и кешу, DNS и вход от шлюза; строка T12 таблицы угроз закрыта.

Лабораторная работа 7. CD и GitOps

Цель

Упаковать приложение в Helm-чарт и организовать доставку, при которой кластер приводится к состоянию репозитория окружений: сборка основной ветки попадает в `dev`, релизный тег — в `prod`, секреты хранятся запечатанными, откат делается через историю Git.

Исходные данные

- инструмент доставки варианта — **Argo CD**, движок конвейера — **GitLab CI**, registry — **node-1:5050**;
- окружения — namespace `dev` и `prod` одного кластера;
- способ хранения секретов один для всех вариантов: Sealed Secrets.

Предусловия

- приложение работает в кластере по манифестам ЛР6;
- конвейер публикует подписанные образы с тегами по SHA и по релизному тегу.

Задание

Чарт и репозиторий окружений

1. Перенесите манифесты ЛР6 в Helm-чарт `deploy/chart` репозитория приложения: шаблоны, `values.yaml` со значениями по умолчанию, отдельные файлы значений для `dev` и `prod` (число реплик, ресурсы, имена хостов маршрута). В `dev` — одна реплика, в `prod` — две; окружения различаются и именами: `dev` отвечает на `dev.node-1` и `dev.node-2`, `prod` — на `prod.node-1` и `prod.node-2`. Чарт должен проходить `helm lint`; значения окружения, включая тег образа, контроллер берёт из репозитория окружений (шаг 2).
2. Создайте на платформе репозиторий окружений с каталогом на окружение. В нём лежат значения окружения и запечатанные секреты; манифесты рендерятся из чарта. Защитите его основную ветку: прямой push разрешён только конвейеру.
3. Заведите для конвейера доступ на запись только к репозиторию окружений и положите его в переменные CI. Конвейер после публикации образа обновляет значения: сборка основной ветки — тег по SHA в каталоге `dev`, релизный тег `vX.Y.Z` — в каталоге `prod`. Ручных шагов между тегом и `prod` быть не должно.
   Доступ — токен проекта (project access token) с областью `write_repository`.
4. Установите контроллер Sealed Secrets в кластер, запечатайте секреты приложения `kubeseal` и положите в репозиторий окружений только запечатанные файлы. Открытых значений в Git быть не должно.

Доставка

5. Настройте доставку инструментом варианта.

   Argo CD ставится в кластер и читает репозитории окружений и приложения по deploy key — ключам SSH, зарегистрированным в настройках каждого без права записи; ключ хоста платформы добавляется в известные Argo CD. Заводятся два `Application` — на `dev` и на `prod` — с автоматической синхронизацией, `prune` и `selfHeal`: без последнего правка объекта руками в кластере не отменяется. Установку от обновления Argo CD не отличает: хук Helm `pre-upgrade` срабатывает и на первой синхронизации, раньше объектов чарта. Порядок миграции задаётся волнами синхронизации (`argocd.argoproj.io/sync-wave`): база, ServiceAccount и секреты, затем Job миграции, затем `Deployment`. Учётных данных кластера в конвейере нет: состояние забирает контроллер.

6. Проверьте доставку до сдачи: релизный тег `vX.Y.Z` доезжает до `prod` без ручных шагов и виден в ответе `http://prod.node-1/version`, сборка основной ветки — до `dev` и видна в `http://dev.node-1/version`.
7. Проверьте откат: `git revert` коммита в репозитории окружений возвращает предыдущую версию в кластере.
8. Проверьте, что ручная правка объекта в кластере не переживает согласование: контроллер или следующий `converge` возвращает состояние из Git.
9. Закройте строки T10b, T11b, T13 и T17 таблицы угроз.

Дополнительное задание

Одно решение на выбор:

- выкатка canary или blue-green с наблюдаемым переключением трафика;
- политика допуска (Kyverno), блокирующая запуск неподписанного образа, — она закрывает строку T8b таблицы угроз.

Приёмка

Основное задание:

- [ ] **показывается на защите** — `revert` коммита в репозитории окружений откатывает релиз;
- [ ] **показывается на защите** — доступ доставки к кластеру ограничен: у pull-варианта в конвейере нет kubeconfig, а контроллер читает репозиторий окружений по deploy key без права записи; у push-варианта — токен `ServiceAccount` с правами только на namespace; студент показывает, где лежат учётные данные, и объясняет, чем deploy key отличается от ключа пользователя;
- [ ] **проверяется по файлам** — чарт со значениями для `dev` и `prod` проходит `helm lint`;
- [ ] **проверяется по файлам** — репозиторий окружений обновляется конвейером по токену или ключу, ограниченному этим репозиторием, его основная ветка защищена;
- [ ] **проверяется по файлам** — кластер приведён к состоянию репозитория окружений контроллером варианта либо `converge` из конвейера;
- [ ] **проверяется по файлам** — новый тег доезжает до `prod` без ручных шагов и виден в `/version`, сборка основной ветки доезжает до `dev`;
- [ ] **проверяется по файлам** — секреты приложения лежат в репозитории окружений только запечатанными;
- [ ] **проверяется по файлам** — строки T10b, T11b, T13 и T17 таблицы угроз закрыты;
- [ ] объяснить принятые решения и повторить любой шаг по требованию.

Дополнительное задание:

- [ ] **проверяется по файлам** — выполнено одно из решений: переключение трафика наблюдаемо, либо неподписанный образ отклонён политикой допуска (строка T8b закрыта), либо секреты расшифровываются контроллером из Git.

Лабораторная работа 8. Наблюдаемость и надёжность

Цель

Развернуть стек наблюдаемости через GitOps, инструментировать приложение и базу данных, сформулировать SLO и алерт, перевести базу данных под оператор и разобрать поведение системы при отказе узла.

Исходные данные

- хранилище метрик и логов варианта — **VictoriaMetrics и VictoriaLogs**, сбор сигналов — **штатные агенты стека**, инструмент доставки — **Argo CD**;
- оператор базы данных один для всех вариантов — CloudNativePG.

Предусловия

- приложение доставляется в `dev` и `prod` через GitOps (ЛР7);
- стек ставится тем же способом: манифесты стека лежат в репозитории окружений, в кластер их приводит инструмент доставки варианта;
- чарт стека метрик содержит объёмные описания ресурсов, поэтому синхронизация выполняется в режиме `ServerSideApply`; иначе Argo CD упирается в предел размера аннотации;
- если сдано дополнительное задание ЛР6, сетевые политики закрывают трафик между namespace: потребуются правила, разрешающие агенту читать метрики приложения, а хранилищу на `node-1` — принимать сигналы с `node-2`.

Задание

Стек и сигналы

1. Опишите в репозитории окружений стек наблюдаемости в namespace `obs` и включите его в доставку. Профиль минимальный: по одной реплике каждого компонента, однобинарный режим (все компоненты системы в одном процессе), хранение метрик и логов — от трёх до семи суток.

   Компоненты: VictoriaMetrics в однобинарном режиме, vmalert (вычисляет правила алертов), Alertmanager, VictoriaLogs, Grafana.

2. Хранилище и алертинг размещаются на `node-1` (`nodeSelector`): их тома привязаны к узлу, и на `node-2` они выключились бы вместе с ним.

3. Разверните агент сбора сигналов на обоих узлах.

   Агенты стека: сбор метрик и доставка логов штатными компонентами варианта.

4. Соберите метрики узлов, состояния кластера и приложения (`/metrics`). Метрики базы данных добавляются на шаге 9: их отдаёт экспортер оператора, а до него база остаётся `StatefulSet` из ЛР6 без экспортера. Постройте в Grafana дашборд RED для приложения: частота запросов, доля ошибок, распределение латентности. Служебные маршруты `/healthz`, `/readyz` и `/metrics` из панелей исключаются: у простаивающей реплики пробы — единственный трафик, и неготовность при недоступной базе превратилась бы в долю ошибок. Дашборд хранится в Git.

5. Настройте сбор логов с обоих узлов. Проверьте поиск по namespace и по приложению и добавьте на дашборд панель ошибок.

6. Проверьте, что в логах нет секретов и данных предметной области: выполните поиск по строке подключения, токенам и содержимому запросов. Найденное устраняется настройкой полей лога. Закройте Grafana аутентификацией: анонимный доступ выключен, пароль администратора приходит запечатанным секретом.

Цели уровня обслуживания и алерт

7. Опишите SLO приложения в файле `SLO.md` репозитория `infra`: SLI (доля успешных запросов и доля запросов быстрее порога), цель на окно 30 суток и бюджет ошибок с арифметикой — сколько минут недоступности допускает цель. Назовите, какие запросы в SLI не входят: служебные маршруты и ответы `404` на несуществующие объекты — с точки зрения доступности успешные.
8. Заведите алерт на долю ошибок с коротким интервалом ожидания (`for` не больше 30 секунд); правило считает долю по тем же запросам, что панели дашборда, без служебных маршрутов. Напишите к алерту runbook в файле `RUNBOOK.md` репозитория `infra` и сошлитесь на него в аннотации алерта: признак, первая проверка, вероятная причина, порядок действий. Проверьте срабатывание, включив инъекцию ошибок:

```
curl -fsS -X POST http://prod.node-1/admin/inject \
  -H "X-Admin-Token: $ADMIN_TOKEN" -d '{"error_rate":0.5,"delay_ms":0}'
```

Значение `ADMIN_TOKEN` — из `Secret` приложения (ЛР6).

Надёжность

9. Переведите базу данных окружения `prod` под оператор CloudNativePG: оператор — в своём namespace, объект `Cluster` — в `prod`, кластер из двух экземпляров, новый секрет подключения, повторный запуск `Job` миграции. Перенос данных не требуется; база `dev` остаётся `StatefulSet` из ЛР6.
10. Добейтесь, чтобы приложение переживало отказ узла: две реплики разнесены по узлам (`podAntiAffinity`), у приложения есть `PodDisruptionBudget`.
11. Проведите учебный отказ дома и ведите журнал с метками времени: выключите `node-2`, зафиксируйте срабатывание алерта и доступность приложения на `http://prod.node-1/`, верните узел и зафиксируйте восстановление. Штатная эвакуация подов с недоступного узла занимает около пяти минут — это часть журнала, а не ошибка. Напишите постмортем на страницу: что произошло, как обнаружено, что восстановило работу, что изменить.
12. Посчитайте пять метрик DORA по своему стенду и запишите их разделом «DORA» в `SLO.md`: частота развёртываний — число релизных тегов, доехавших до `prod` за семестр; время от коммита до `prod` — для последнего релиза, от коммита тега до синхронизации контроллером (у `werf` — до завершения конвейера доставки); время восстановления — для отката ЛР7, от появления откатываемой версии в кластере до возврата предыдущей (у Argo CD и Flux — по истории синхронизаций, у `werf` — по журналу конвейера окружений); доля неудачных изменений — откаты ЛР6 и ЛР7 к числу выкаток; доля переделок — внеплановые выкатки, исправлявшие отказ в `prod`, к числу выкаток.
13. Закройте строки Т14, Т15 и Т16 таблицы угроз.

Дополнительное задание

Разверните хранилище трейсов (Tempo или Jaeger), включите приложению экспорт по OTLP и свяжите трейс с логами по `request_id`.

Приёмка

Основное задание:

- [ ] **показывается на защите** — логи ищутся по namespace и по приложению, на дашборде есть панель ошибок;
- [ ] **показывается на защите** — после включения инъекции ошибок алерт на долю ошибок срабатывает не позже чем через минуту;

- [ ] **проверяется по файлам** — хранилище варианта и Grafana установлены через GitOps в минимальном профиле, хранилище и алертинг размещены на `node-1`, агент работает на обоих узлах;
- [ ] **проверяется по файлам** — собираются метрики узлов, состояния кластера, приложения и базы данных, дашборд RED хранится в Git;
- [ ] **проверяется по файлам** — в логах приложения нет секретов и данных предметной области (проверено поиском), Grafana закрыта аутентификацией;
- [ ] **проверяется по файлам** — `SLO.md` содержит SLI, цель и бюджет ошибок с арифметикой; на алерт доли ошибок написан runbook;
- [ ] **проверяется по файлам** — раздел «DORA» в `SLO.md` содержит пять метрик, посчитанных по стенду, с указанием источника каждой;
- [ ] **проверяется по файлам** — база данных окружения `prod` работает под оператором в двух экземплярах, у приложения есть `PodDisruptionBudget`, две реплики разнесены по узлам;
- [ ] **проверяется по файлам** — журнал учебного отказа с метками времени и постмортем на страницу;
- [ ] **проверяется по файлам** — строки T14, T15 и T16 таблицы угроз закрыты;
- [ ] объяснить принятые решения и повторить любой шаг по требованию.

Дополнительное задание:

- [ ] **проверяется по файлам** — трейсы приложения попадают в хранилище трейсов и связаны с логами по `request_id`.